Formal Verification of Imperative Higher-Order Functions

Wolfgang Grieskamp* and Teng Zhang and Vineeth Kashyap and Jake Silverman
Aptos Labs, Palo Alto, USA
*Corresponding author: wg@aptoslabs.com

***Abstract*—The Move Prover (MVP) is a formal verifier for smart contracts written in the Move programming language. Recently, Move on Aptos was extended with *higher-order functions*: function values that can be passed around, stored in data structs, and kept in persistent storage, enabling *dynamic dispatch*. This paper describes the novel representation of function values in the Move specification language and their implementation in MVP. We introduce *behavioral predicates* which characterize Move functions (aborts and pre/post conditions) by single-state or two-state predicates. We also introduce *state labels* for naming intermediate memory states in which expressions are evaluated and which allow to compose behavioral predicates to describe sequences of state transitions. On SMT level, function values are encoded by discriminating over the possible function values reaching a call site: when the concrete function is known, its effect is accounted for directly; when it is unknown (for example, a function parameter, or a closure loaded from storage), its behavioral predicates describe the effect. We are able to automatically validate completeness and correctness of the AI-assisted implementation by using *specification inference*: by synthesizing specs of arbitrary code with higher-order functions and then running MVP on the result, a self-verifying feedback loop is realized, which significantly improved the ability of AI to support this work.**

I. Introduction

The Move Prover (MVP) [1] is a formal verification tool for Move smart contracts that is designed to be used routinely during code development, with a finish time comparable to type checkers and linters. MVP is deployed at Aptos [2] for formal verification of core protocol logic such as staking, metering, code deployment, and auxiliary data structures that underpin the framework. It has been originally developed for the Libra project at Meta, where it was used to specify the Diem payment network.

Recently, the Move language was extended with *higher-order functions* (HO) [3]. In *Move 2*, functions are first-class values: they can be passed as arguments, returned from functions, stored in struct fields, and persisted in global storage. Combined with Move's resource model, persisted function values give the developer a principled account of *dynamic dispatch*. Typical use cases on Aptos include callback-based asset flows, extensible asset managers, dispatchable permissions and metering, and generic data structure operations parameterized by a caller-provided function (e.g. fold, map, filter, custom comparators).

Higher-order functions are a known challenge for formal verification. They break the modular, static-dispatch specification style on which MVP was built. When a function is dispatched dynamically, its pre- and post-conditions cannot in general be looked up at the call site. Instead, the caller's specification must refer *abstractly* to the behavior of its function-value arguments, and the verifier must reconcile this abstract specification with whichever concrete function happens to reach the call site. For a production-oriented prover, this reconciliation must be fast, complete in the relevant cases, and free of the trigger instability that higher-order quantification easily introduces [4].

Contributions. This paper describes the extension of MVP to support higher-order Move, and the underlying extension of the Move specification language: Specifically:

- We extend Move specifications with *behavioral predicates*: $\mathsf{requires_of}\langle f\rangle(\bar{x})$, $\mathsf{aborts_of}\langle f\rangle(\bar{x})$, and $\mathsf{ensures_of}\langle f\rangle(\bar{x})$, which characterize the pre-/abort-/post-behavior of a function value f on arguments $\bar{x}$, enabling reasoning about function specifications as first-class citizens.
- We introduce *state labels* into Move specifications, which name intermediate memory states along a function's execution and connect them to the pre- and post-state via two-state predicates, as in $(..S \mid\!\sim \phi) \wedge (S.. \mid\!\sim \psi)$. This enables reasoning about sequential evolution of global state in predicates.
- We describe how MVP implements invocation of function values on top of SMT [5], [6], by discriminating over the possible types of function values flowing into the call site: if f is known (a ground constructor), the invocation is replaced by the effect of the concrete function; if f is not known (for example, a function-typed parameter or a closure loaded from storage), the behavioral predicates of f describe its effect.
- We validate the design by extending MVP's *specification inference* tool to function-valued code. With behavioral predicates as the abstraction, the tool computes weakest-preconditions over Move bytecode that contains closures and dynamic dispatch. Feeding those generated specs back into the verifier validates completeness and correctness of the solution.

The remainder of the paper is organized as follows. Sec. II reviews Move on Aptos, introduces the running automated-market-maker (AMM) example, and illustrates common specification patterns using function values. Sec. III describes the encoding of behavioral predicates and state labels, and

https://doi.org/

This article is licensed under a Creative Commons Attribution 4.0 International License

describes the Boogie encoding of dynamic dispatch in MVP. Sec. IV reports on validating the encoding via the specification inference tool. Sec. V discusses related work and concludes.

II. Using Function Values in Move Specifications

This section recalls the aspects of Move on Aptos that matter for the verification of higher-order programs, introduces an automated market maker (AMM) as a running example, and illustrates the common specification patterns our extension supports.

A. Move on Aptos

Move [7], [8] is a bytecode-verifiable programming language designed from the outset to coexist with formal verification. For the purposes of this paper, the relevant properties are:

- *Strong static typing with generics.* Every expression has a statically known explicit type; generic functions and structs are monomorphized for verification [1].
- *Reference semantics à la Rust.* References are borrowed from local variables or from global storage and cannot be stored inside structs that go to global storage; this yields an alias-free memory model that makes verification tractable [1].
- *Type-indexed global storage.* Global state is a map keyed by (resource type, address) pairs: a struct with the `key` ability can be published, borrowed, and moved at an address via `T[addr]`, with linear ownership enforced statically. Specifications quantify over this memory directly, and `modifies` clauses name the footprint a function may touch, providing a frame condition.
- *Strongly-typed bytecode.* Move source is compiled to bytecode whose type- and memory-safety are checked before execution. MVP verifies this bytecode against specifications in the source.
- *Integrated specification language.* Move lets developers write pre-/post-conditions, abort conditions, data invariants, and global memory invariants in the same source file as the code they describe, or optionally, into separate files.

Function values in Move 2. Move 2 extends the type system with *function types* $T_1 \times \cdots \times T_n \to T$ (written `|T1,...,Tn|T` in the concrete syntax) and allows values of such types to be constructed from concrete function symbols or from lambda expressions, possibly with captured arguments. In Move's type ability system, function values may be marked `copy`, `drop`, and `store`; a function value with `store` can be persisted to global storage. Invocation has the form `(expr)(args)` and is dispatched based on the closure reaching the call site.

B. Automated Market Maker Example

The running example throughout this paper is a simplified AMM module (Uniswap v2 [9]) in which the *pricing curve* is a function value stored in the pool. The pricing function has to satisfy a number of invariants: (i) the function never aborts, (ii) the output never exceeds the output reserve, (iii) the output is monotone in the input amount, and (iv) the product of the two reserves does not decrease after a swap (constant-product preservation).

```
struct Pool has key {
  reserve_x: u64, reserve_y: u64,
  // (reserve_in, reserve_out, amt) = out
  pricing: |u64, u64, u64|u64 has copy, store
}
spec Pool {
  reads_of<pricing> *;

  // Pricing must never abort.
  invariant ∀ S in *, ri: u64, ro: u64, a: u64:
    S |~ !aborts_of<pricing>(ri, ro, a);

  // Output never exceeds the output reserve.
  invariant ∀ S in *, ri: u64, ro: u64, a: u64:
    S.. |~ result_of<pricing>(ri, ro, a) ≤ ro;

  // Monotonicity in the input amount.
  invariant ∀ S in *, ri: u64, ro: u64,
                          a1: u64, a2: u64:
    S.. |~ a1 ≤ a2 ⟹
      result_of<pricing>(ri, ro, a1)
            ≤ result_of<self.0>(c, ri, ro, a2);

  // Constant-product preservation.
  invariant ∀ S in *, ri: u64, ro: u64, a: u64:
    S.. |~ (ri + a)
           * (ro - result_of<pricing>(ri, ro, a))
                 ≥ ri * ro;
}
```

Each invariant is universally quantified over a starting state `S` in which the pricing function is evaluated. We write `S |~ p` for a single-state predicate to be evaluated in state `S`, and `S.. T |~ q` for a two-state predicate, where the pre/post state can also be omitted (as in `S.. |~ q` in the example). Note that state symbols like `S` are labels representing a set of global resources present at a given program point. Which resources those are is determined by the `reads_of<f>` clause. In the example, the pricing function is allowed to read any state relevant for the verified program segment.

The behavioral predicates `aborts_of`, `result_of`, and (elsewhere) `ensures_of` and `requires_of` take a function value and its arguments and evaluate to the corresponding aspect of the call's behavior. Using them in invariants as in the example creates (i) a requirement for any code creating a pool to pass only compliant function values, and (ii) a guarantee for any code calling a pool function that the requirements are met.

Swapping. The major functionality of the pool is to facilitate `swap` (exchange) of one asset with another.

```
fun swap(pool: &mut Pool, amt: u64): u64 {
  let out = (pool.pricing)(
              pool.reserve_x, pool.reserve_y, amt
                );
  pool.reserve_x += amt; pool.reserve_y -= out;
  out
}
spec swap {
  // Can only abort on reserve overflow.
  aborts_if pool.reserve_x + amt > MAX_U64;
  // Outcome is determined by pricing at pre-state
      reserves.
  ensures result = old(result_of<pool.pricing>(
```

```
            pool.reserve_x, pool.reserve_y, amt));
  // Preserves product in pool.
  ensures pool.reserve_x * pool.reserve_y
          ≥ old(pool.reserve_x * pool.reserve_y)
              ;
}
```

The specification of swap can be verified by MVP because of the invariants guaranteed for the (dynamic) pricing function. In this case, we only illustrate the aborts condition and product preservation. For aborts, pricing itself is non-aborting, so only the overflow conditions remain. (Notice MVP would flag any aborts conditions not covered). For product preservation, the constraint can be derived as well.

Pricing variants. At the time a pool value is constructed, the invariants for the pricing function need to be verified. Obviously this is more difficult than the application side, since now properties of functions need to be proven. In reality, those proofs will often need custom solutions, but for the given example, MVP can handle some cases gracefully. First, we define a function which computes the so-called *constant product*, which is a pricing function describing a direct swap without fees; however, rounding errors can lead to a loss in returned assets:

```
fun product(ri: u64, ro: u64, a: u64): u64 {
  let num = (ro as u128) * a;
  let den = (ri as u128) + a;
  if (den = 0) 0 else (num / den) as u64
}
```

When constructing a pool using Pool{.., pricing: product}, MVP is able to discharge all verification conditions.

The next example uses a pricing function which charges a configurable fee:

```
struct Fee has key {bps: u64}
fun with_fee(o: address,
             ri: u64, ro: u64, a: u64): u64 {
  let b = (a as u128)
     * (10000 - Fee[o].bps as u128) / 10000;
  product(ri, ro, b)
}
```

In order to construct a pricing function we need to curry the address for the fee resource and capture it in a closure, which can be done in Move as below:

```
  let o: address;
  Pool{
    ..,
    pricing: |ri, ro, a| with_fee(o, ri, ro, o)
  }
```

MVP detects that with_fee violates the requirement to not abort. (Abort happens if the resource Fee[o] does not exist.) Guarding the definitions with checks and using a default fee fixes this and makes verification succeed.

The AMM example shows that non-trivial invariants can be specified for function values, and that the prover is capable of verifying them on construction and caller side. What has been shown for data field invariants carries over to function parameters and function call sites. The invariants presented constitute non-linear arithmetic problems, which are notoriously hard to solve. In the full example, Move 2 proof hints are used to support reliable verification of problems of this kind.

C. Filter-style Loops

Behavioral predicates are not limited to struct invariants; they appear naturally in loop invariants and function specs for higher-order collection operations. As a minimal illustration, consider a generic find which returns the first index of v whose element satisfies a predicate pred, or len(v) when no such element exists. A spec-level helper no_match_before captures the recurring "no earlier element has matched" pattern:

```
fun find<T>(v: &vector<T>,
            pred: |&T|bool has copy + drop): u64 {
  let i = 0; let n = vector::length(v);
  while (i < n) {
    if (pred(vector::borrow(v, i))) return i;
    i = i + 1;
  } spec {
    invariant i ≤ n;
    invariant no_match_before(v, pred, i);
  };
  n
}
spec find {
  ensures result ≤ len(v);
  ensures no_match_before(v, pred, result);
  ensures result < len(v)
            ⟹ result_of<pred>(v[result])
}
spec fun no_match_before<T>(v: vector<T>,
                pred: |&T|bool, end: u64): bool {
  ∀ j in 0..end: !result_of<pred>(v[j])
}
```

no_match_before is an ordinary spec function, but its body calls the behavioral predicate result_of<pred> on each element — a predicate-parametric abstraction is thus first-class in Move and can be reused across loop invariants and post-conditions. The function's post-condition characterises result entirely through the same evaluator: either result is a valid index whose call is the first to match, or result = len(v) and no element matches at all. No concrete predicate is ever named in either the body or the spec; a caller instantiates pred with any function value, and the specification is discharged through its behavioral contract.

D. More About State Labels

Universally quantified state labels have been used for specifying the invariants of the pricing functions. Another major use case of state labels is to express intermediate states resulting from sequential execution in Move. One can define a higher-order function which composes the execution of two (stateful) other functions:

```
fun followed_by(f: |&mut A|, g: |&mut A|, x: &mut
    A) {
  f(x);
  g(x)
}
spec followed_by() {
  ensures ∃ S in *:
    (..S |~ ensures_of<f>(x)) &&
    (S.. |~ ensures_of<g>(x))
}
```

The spec denotes that the ensures of f is evaluated from the pre-state of followed_by into S (..S), and the ensures of g

from S to the post-state (S..). Thus the post state of x resulting from the ensures_of<f> becomes the pre-state (the old(x)) of the ensures_of<g>

The application of state labels in this context is not restricted to higher-order functions; concrete function names can be passed to behavioral predicates as well. This allows to compose the specs of wrappers around other functions via behavioral predicates of those functions, without the need to duplicate specs. Just consider the above example, but f and g replaced by concrete functions.

III. Encoding

We describe the implementation of function values as a translation into an imperative intermediate verification language (IVL) inspired by Boogie [10]. The encoding is given at the IVL level because this is where the interesting design decisions become visible — the subsequent step to pure SMT is straightforward and uses standard techniques. The fragment of the IVL used in this section is familiar: procedures with **havoc**, **assume**, **assert**, pure functions, *algebraic datatypes* (tagged unions with selectors, like Rust enums), universal *axioms* with *triggers*, and polymorphic map types. State is represented by Boogie-style maps; in particular, each resource type τ yields a map $M_\tau \in \mathsf{address} \to \tau$, with address being the domain of addresses in Move (256-bit integers). The representation is similar to that in the earlier MVP paper [1].

In the following we focus on the aspects specific to function values: the representation of function values (Sec. III-A), the encoding of invocation of function values, the encoding of behavioral predicates via axioms (Sec. III-D), how the state labels of Sec. II materialize as extra memory parameters (Sec. III-C), and finally the frame-condition (Sec. III-E). Everything in this section operates *after* monomorphization, so type parameters have been specialized and the set of closure, parameter, and field variants reaching a given function type is known statically [1].

A. Representing Function Values

For each function type $\tau = (T_1, \ldots, T_n) \to T$ that occurs in the monomorphized program, we create a datatype in the IVL which represents all the potential *sources* of function values in a given program. To this end, MVP's mono-analysis records three sets: the set $\mathcal{C}_\tau$ of concrete functions f constructing closures of type τ, the set $\mathcal{P}_\tau$ of function-typed parameters of verification targets, and the set $\mathcal{F}_\tau$ of storable function-typed struct fields whose type is τ. The union of these three sets is materialized as an algebraic datatype in the IVL — one constructor per variant:

$$\begin{array}{l}\textbf{datatype}\ \tau\ \{\\ \quad C_f(\overline{c:T}), \ldots, P_{g,x}(), \ldots, F_{S.x}(n:\mathbb{N}), \ldots\\ \}\end{array}$$

where $\overline{T}$ are the parameter types of f that are captured by a closure. Parameter variants are nullary because, in a well-typed VC, the value of an entrypoint function-typed parameter is a skolem constant pinned to its variant at entry (see below); no runtime payload is required. Field variants carry an integer discriminator n so that two locations in memory holding distinct field values are not forced to be equal — datatypes with a single nullary constructor would otherwise collapse.

Notice that this representation captures *all* possible functions of type τ relevant for a given VC. So we do not actually need to deal with an arbitrary number of function values when it comes to higher-order function verification, but only those which can be derived from the program. The representation is further narrowed if the function value of a given type is not stored in a structure which ends in global memory. Storing a function value amounts to dynamic dispatch (think of building a 'vtable' explicitly), and this additional complexity is reflected by the n in $F_{S.x}(n)$. Nevertheless, we are able to pinpoint a specification to $F_{S.x}(n)$, as has been seen for the AMM example (cf. Sec. II-B).

B. Invoking Function Values

When a function value is invoked, we are looking at either a concrete closure value C_g or a parameter or field selection as described above. In the former case, one can simply delegate to calling the actual underlying function f, by composing the captured and provided arguments into a full argument list.

In the other cases of parameters and field selections, we use the behavioral predicates as defined for the function value, using the standard schema for encoding a call to a Move function via the function's specification. This reduces the problem of invocation to how those behavioral predicates are encoded.

$$\begin{array}{l}\textbf{proc}\ \mathit{invoke}_\tau(x:\tau, \overline{p:T})\ \mathsf{returns}\ (\overline{r:T})\\ \quad \textbf{if}\ x\ \mathsf{is}\ \ C_f(\overline{c})\\ \qquad\quad \overline{r} := \mathsf{call} f(\overline{c}, \overline{p})\\ \quad \textbf{else}\\ \qquad\quad \textbf{assert}\ \mathit{requires}_\tau(x, \mathcal{M}_\tau, \overline{p})\\ \qquad\quad \textbf{if}\ \ \mathit{aborts}_\tau(x, \mathcal{M}_\tau, \overline{p})\\ \qquad\qquad\quad \mathit{abort_flag} := \textbf{true}\\ \qquad\quad \textbf{else}\\ \qquad\qquad\quad \textbf{let}\ \mathcal{M}'_\tau := \mathcal{M}_\tau\\ \qquad\qquad\quad \textbf{havoc}\ \Delta_\tau(\overline{p}),\ \overline{r}\\ \qquad\qquad\quad \textbf{assume}\ \mathit{ensures}_\tau(x, \mathcal{M}'_\tau, \mathcal{M}_\tau, \overline{p}, \overline{r})\end{array}$$

See below for $\mathcal{M}$ and Δ definitions. Notice that the representation of the else branch is very similar to how opaque functions are mapped down to the solver. In fact, for an opaque function and provided $\mathit{len}(c) = 0$, the if-else branches are semantically identical. The translation of the IVL to SMT takes care of converting the $\mathsf{call} f$ of an opaque f into exactly what we find in the else branch here, so we can omit this detail.

C. Computing Memory Accesses

In order to express invoke_τ, we need to reason about the memory usage of a function value type τ, $\mathcal{M}_\tau$, as well as that subset of memory locations which are modified, $\Delta_\tau(\overline{p})$. To recall, in the Move language, persistent global storage, or memory, can be accessed via type indexing. For instance, if R is a resource type, we write &mut R[addr] in the language to

obtain a mutable reference to the memory which can then be subsequently mutated.

MVP's representation of memory uses type-indexed maps $M_\rho \in \mathsf{address} \to \rho$, with ρ a resource type (a `struct` in Move with the `key` ability). In the imperative procedures of the IVL, those maps are implicitly available as global variables. But in pure expressions like behavioral predicates, they need to be made explicit as parameters; this is the reason why we have been passing $\mathcal{M}_\tau$ to behavioral predicates earlier.

A few definitions for memory maps are needed. Note that these are definitions on the meta level of the description here and not part of the IVL; they are used to describe pseudo code in terms of the IVL.

First, with $\mathcal{M}_R = \{M \in \mathsf{address} \to \rho \mid \rho \in R\}$ we denote a set of memory maps for the resource types in R. The R can be omitted if it is clear from the context. For convenience, with $\mathcal{T}(\mathcal{M})$ the set of all types ρ used in the memory maps of $\mathcal{M}$ is described. With $\mathcal{M}_R \downarrow S$, where $S \subseteq R$, the projection $\{M \in \mathcal{M}_R \mid \rho(M) \in S\}$ is denoted.

In addition to memory, we also need to reason about modifications. A modification is a set of pairs of a type and an expression of the IVL of type address, $\Delta \in \mathbb{P}(\mathcal{T} \times \mathcal{E})$. We write $\Delta(\overline{p})$ for a set of modifications built on expressions referencing parameters in $\overline{p}$.

General Access. Memory access $\mathcal{M}_\tau$ is computed as the union of all the accesses in the variants of τ, $\mathcal{M}_{C_f} \cup \mathcal{M}_{P_{f,x}} \cup \mathcal{M}_{F_{S.x}}$. For a closure, $\mathcal{M}_{C_f}$ can be statically derived from the code. For parameters and fields, the information is declared in the spec block:

```
spec foo(f: |T|S, g: |T|S) {
  reads_of<f> R; // f can read resource R
  reads_of<g> *; // g can read any resource
                 // in scope of the VC
}
```

For the wildcard operator `*`, all memory which is read or written as part of the currently generated VC is included, and in addition an abstract memory map for any 'unknown' memory the function may depend on. The unknown memory is needed for a sound encoding. For instance, if we verify function `foo` above in a context where there is no memory involved anywhere, we must not assume that `g(x)` delivers the same value in different states, as it may depend on this unknown additional memory. However, we can still know that the specification of `g` holds in every possible state.

Write Accesses. Similarly to general access, the modifies set Δ_τ is computed as the union of all the modifications in the variants of τ, $\Delta_{C_f} \cup \Delta_{P_{f,x}} \cup \Delta_{F_{S.x}}$.

For closures, Δ_{C_f} is derived from the code and, if present, the `modifies R[a]` declaration in the language. These existed before higher-order functions, for specifying what functions modify.

For parameters and fields, the information is declared in the spec block, where for the `modifies_of<f>` clause, the parameters of the function value itself are passed:

```
spec foo(f: |address|) {
  // f can modify resource R[a]
  modifies_of<f>(a: address) R[a];
}
```

If no `modifies_of` is declared, it is assumed that the function does not modify global memory.

Notice that a wildcard (`modifies_of<f> *`) is not supported, since the invocation of `f` would render the memory as arbitrary (no effective frame condition). This is a notable restriction on what we can specify with higher-order functions in Move and is left for future work.

D. Behavioral Predicates

Behavioral predicates are derived by switching over the variants of the function value representation τ, similarly to the *invoke*$_\tau$ procedure above, delegating extraction of the underlying predicates to per-variant functions.

Below, the definition for ensures is given. The other predicates are defined similarly. Specific to ensures is that it needs to take care of the frame condition: the caller havoced the full modification set, but only a subset of that is constrained by the given variant. The handling needs to account for that addresses in Δ are only symbolic expressions with unknown aliasing:

$$
\begin{array}{l}
\textbf{fun}\ \mathit{ensures}_\tau[\mathcal{M}', \mathcal{M}](x : \tau, \overline{p : T}, \overline{r : S}) : \mathsf{bool} \\
\quad \bigwedge_{(\rho,a) \in \Delta_\tau} (\\
\qquad \textbf{if}\ \rho \notin \mathcal{T}(\Delta_x) \\
\qquad\qquad \mathcal{M}(\rho) = \mathcal{M}(\rho') \\
\qquad \textbf{else} \\
\qquad\qquad (\bigwedge_{(\rho,a') \in \Delta_x} a' \neq a) \\
\qquad\qquad\qquad \Rightarrow \mathcal{M}(\rho)[a] = \mathcal{M}(\rho')[a] \\
\quad) \wedge \mathit{ensures}_x[\mathcal{M}'_x, \mathcal{M}_x](p, r)
\end{array}
$$

The relational representation of a function via ensures does not guarantee proper functional behavior. This is achieved by introducing *result*$_\tau$ as an uninterpreted function, connected via an axiom:

$$
\begin{array}{l}
\textbf{fun}\ \mathit{result}_\tau[\mathcal{M}', \mathcal{M}](x : \tau, \overline{p : T}) \to \overline{S} \\
\textbf{axiom}\ \forall x, \overline{p}, \overline{r} \bullet \\
\quad \mathit{ensures}_\tau[\mathcal{M}', \mathcal{M}](x, \overline{p}, \overline{r}) \Leftrightarrow \overline{r} = \mathit{result}_\tau[\mathcal{M}', \mathcal{M}](x, \overline{p})
\end{array}
$$

The actual conditions extracted depend on the variant. For closures, they are directly extracted from the spec block. For parameters and fields, we introduce uninterpreted functions.

$$
\begin{array}{l}
\textbf{fun}\ \mathit{ensures}_{C_f(\overline{c})}[\mathcal{M}', \mathcal{M}](\overline{p : T}, \overline{r : S}) : \mathsf{bool} \\
\quad \langle \mathit{extract\ from\ spec\ block\ of}\ f(\overline{c}, \overline{p}) \rangle \\
\textbf{fun}\ \mathit{ensures}_{P_{f,x}}[\mathcal{M}', \mathcal{M}](\overline{p : T}, \overline{r : S}) : \mathsf{bool} \\
\textbf{fun}\ \mathit{ensures}_{F_{S.x}}[\mathcal{M}', \mathcal{M}](\overline{p : T}, \overline{r : S}) : \mathsf{bool}
\end{array}
$$

The meaning of the uninterpreted functions is determined by how they are injected into a VC via the regular mechanism by which pre/post conditions and invariants are handled. Consider a precondition like `requires aborts_if<param>(x)`, where `param` is a function-value parameter of a function `foo`. This will be injected in the VC as $\textbf{assume} \neg\ \mathit{aborts}_{P_{foo,param}}(x)$. Similarly, for a data invariant of a field an assumption will be generated when the field is selected.

E. Intermediate State Labels

Consider the following specification using intermediate state labels:

```
struct R(u64) has key;
spec foo(f: |address|, g: |address|, a: address) {
  ensures ..S |~ ensures_of<f>(a)
  ensures S |~ R[a].0 > 0;
  aborts_if S |~ aborts_of<g>(a);
  ensures S.. |~ ensures_of<g>(a)
}
```

As outlined in the last section, we want to be able to extract behavioral predicate functions from this specification. For the ensures we can come up with the following, which simply replicates part of the spec:

$$\textbf{fun}\ \mathit{ensures}_{C_{foo}}[\mathcal{M}', \mathcal{M}](a)$$
$$\exists\, \mathcal{M}^S \bullet \mathit{ensures}_{\tau}(f, \mathcal{M}', \mathcal{M}^S, a) \wedge R[\mathcal{M}^S][a].0 > 0 \wedge \mathit{ensures}_{\tau'}(g, \mathcal{M}^S, \mathcal{M}, a)$$

However, for the aborts which happens at the intermediate state `S` the situation is not that straightforward. The state `S` needs to be first established before the aborts condition can be checked for g. To achieve this, one could take all ensures conditions which reach `S` and then evaluate the aborts condition, as in:

$$\textbf{fun}\ \mathit{aborts}_{C_{foo}}[\mathcal{M}', \mathcal{M}](a)$$
$$\exists\, \mathcal{M}^S \bullet \mathit{ensures}_{\tau}(f, \mathcal{M}', \mathcal{M}^S, a) \wedge R[\mathcal{M}^S][a].0 > 0 \wedge \mathit{aborts}_{\tau'}(g, \mathcal{M}^S, a)$$

But this is *not* correct. To see this, consider that f leaves the resource in a state such that $R[\mathcal{M}^S][a].0 > 0$ does not hold. In this case, the $\mathit{aborts}_{C_{foo}}$ above returns false, indicating that *foo* would not abort. Yet it may very well abort if g aborts in $\mathcal{M}^S$.

The problem here is to differentiate between conditions which *define* and which *constrain* intermediate states. The current implementation in MVP employs a heuristic to determine this:

- Two-state behavioral predicates (`ensures_of` and `result_of`) are considered defining predicates. Also, a set of builtin predicates for publishing, updating, and removing resources are in this category.
- Everything else is considered a constraining predicate.

In order to extract the condition for an aborts, MVP will extract all conjuncts in the spec defining the state which reaches the aborts condition, stripping out conditions which are only constraining. This defining fragment of the spec is then combined with the aborts predicate. Practically, for the example above, we get:

$$\textbf{fun}\ \mathit{aborts}_{C_{foo}}[\mathcal{M}', \mathcal{M}](a)$$
$$\exists\, \mathcal{M}^S \bullet \mathit{ensures}_{\tau}(f, \mathcal{M}', \mathcal{M}^S, a) \wedge \mathit{aborts}_{\tau'}(g, \mathcal{M}^S, a)$$

We point out that the heuristic used here is potentially unsound. If $\mathit{aborts}_{C_{foo}}$ underapproximates and returns false where it should not, valid aborts scenarios can be missed in verification. This is a problem which requires future work and refinement.

IV. Validation via Specification Inference

Together with the introduction of behavioral predicates and function values, specification inference based on *weakest precondition analysis* [11] was added to MVP. One can think of WP computation as an inversion of verification: given arbitrary code, specifications are generated which are then supposed to pass verification, and precisely describe the code. This enables a validation pipeline where we go code $\rightsquigarrow$ code+specs $\rightsquigarrow$ VC, with verification expected to succeed. This pipeline is tremendously helpful in AI-supported development, as it gives a baseline to AI to validate correctness of generated or provided code. Tab. I gives an overview of the current behavioral-predicate-related tests using this pipeline — including function value parameters and struct fields.

WP uses behavioral predicates to specify the effect of calls to other functions. This enables formulating the WP of one function in a modular fashion, abstracting from the called function. Here is a simple example:

```
use account::other;
fun incr(x: u64): u64 {
  other::func(x) + 1
}
spec incr {
  pragma opaque;
  ensures [inferred]
    result = result_of<other::func>(x)
}
```

Without behavioral predicates, WP analysis results would have been impractical to represent in Move source code, as we could only inline specs, leading to an explosion of condition sizes.

V. Discussion and Conclusion

A. Related work

Higher-order programs with side effects have been a long-standing concern in program verification. Our approach to behavioral predicates descends from the relational tradition of VDM [12], Z [13], [14], and the B method [15], in which operations are specified by pre/post relations over an explicit state; state labels generalize this to intermediate snapshots and make the relational flavor composable.

Among contemporary verifiers for imperative code with higher-order functions, Verus [16] offers function-value specifications via *spec closures* and phantom ghost state; the approach is state-passing, with state threaded as an extra argument to specifications rather than quantified over directly. Our treatment is more directly relational: state labels denote memory snapshots and are bound by quantifiers in the surface syntax, avoiding the encoding cost of an explicit state argument and matching the style in which Move developers already write pre/post specifications.

F* [17] likewise verifies higher-order, effectful code, internalizing the WP calculus in its type system via Dijkstra monads [18]. Both systems support pre/post specifications, quantifiers, behavioral abstraction over function values, and structured proof guidance — in Move via `proof` blocks, `lemma` declarations, and `calc`, `split`, `apply` statements. F*'s dependent type and effect system goes beyond Move's support,

TABLE I: Inference test categories that exercise behavioral predicates.

Category	Representative cases
Direct calls	Single and chained calls to a spec'd helper; inferred specs use `ensures_of` and `aborts_of` to abstract over the callee's effect rather than inlining its body.
Mutual recursion	Mutually recursive `is_even`/`is_odd`; inference breaks the cycle by phrasing each spec in terms of the other's behavioral predicate.
Higher-order parameters	A function that takes a closure and applies it; the inferred spec abstracts the closure via `ensures_of`/`aborts_of` on the parameter, with no commitment to a particular implementation.
Stored-closure dispatch (enum)	Calculator with an enum variant carrying a closure field; dispatch through the variant is summarised by behavioral predicates on that field.
Stored-closure dispatch (struct field)	Vault with a `Strategy` closure stored in a struct field on Aptos framework types; inference emits `modifies_of` for the field together with the field-form behavioral predicates.
Pre/post state labels	`move_to`, `move_from`, mutable resource indexing; inferred specs distinguish `@pre` from post-state on the memory the predicate observes.
Intermediate state labels	Sequenced state-modifying calls (e.g. remove-then-publish); inferred specs introduce existentially quantified intermediate labels and pin them to defining behavioral predicates.

which is more minimal and focused on Rust-style references and effects on global memory. Move also favors separation of specification from program code, which is useful in environments where work itself is distributed along that axis (for example, a multi-agent system).

Dafny [19] is the closest cousin of MVP: it has a similar specification model of pre/post conditions, higher-order functions, and behavioral predicates. Differences lie in the treatment of state: Move's resource-separated global memory and alias-free mutable reference model versus Dafny's heap model. While both languages support state labels, Move state labels as described here can be used in abstract specifications via universal and existential quantification.

Lean 4 [20] is a dependently-typed interactive theorem prover in the tradition of older similar systems [21], [22], supporting higher-order functions. Verification is tactic-driven and kernel-checked. Lean does not directly work on a programming language as MVP does, but requires the user to create an encoding of the language inside its meta logic. For modeling of stateful systems, one would typically use monads, though any other model can be simulated as well. In contrast, MVP is directly integrated with the Move language and leverages Move's specific memory and reference model. Furthermore, MVP aims at automated SMT-based verification, with optional proof scripts.

In the smart-contract setting, dynamic dispatch is a central challenge for Solidity verifiers. Certora [23] handles dispatch on Solidity by enumerating the possible callees of a virtual call (derived from the contracts in scope) and generating a case-split proof obligation per callee; summaries can be supplied by the user to abstract external calls. Other SMT-based Solidity tools [24], [25], [26] handle dynamic dispatch through similar case-splitting, with varying degrees of abstraction. MVP is more general: a call site's dispatch is explicit in the closure's datatype, and our encoding lets the prover automatically fall back from case-split (when the concrete function is known) to behavioral-predicate reasoning (when only the function type is known), without the user having to structure the specification differently.

B. Future Work

C. Conclusion

We described the extension of the Move Prover to Move 2's higher-order functions and dynamic dispatch. The extension rests on two language additions — behavioral predicates and state labels — and on a Boogie encoding that discriminates closure invocations on the function-value variant reaching the call site. The design preserves MVP's production-oriented cost model — routine, predictable, counterexample-driven verification — in the presence of dynamic dispatch. We validated the approach by extending MVP's specification inference tool to closures and dynamic dispatch, which now computes weakest-preconditions over Move bytecode containing function values.

References

[1] D. L. Dill, W. Grieskamp, J. Park, S. Qadeer, M. Xu, and J. E. Zhong, "Fast and reliable formal verification of smart contracts with the move prover," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2022)*, ser. Lecture Notes in Computer Science, vol. 13243. Springer, 2022, pp. 183–200.

[2] J. Park, T. Zhang, W. Grieskamp, M. Xu, G. D. Giacomo, K. Chen, Y. Lu, and R. Chen, "Securing Aptos Framework with Formal Verification," in *5th International Workshop on Formal Methods for Blockchains (FMBC 2024)*, ser. Open Access Series in Informatics (OASIcs), vol. 118. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 9:1–9:16.

[3] W. Grieskamp, "Move goes higher order," https://medium.com/aptoslabs/move-goes-higher-order-cdf9688a4919, Oct. 2025, aptos Labs blog.

[4] K. R. M. Leino and C. Pit-Claudel, "Trigger Selection Strategies to Stabilize Program Verifiers," in *Proceedings of the 28th International Conference on Computer Aided Verification, Part I*. Springer, 2016, pp. 361–381.

[5] C. Barrett, A. Stump, and C. Tinelli, "The SMT-LIB Standard: Version 2.0," in *Proceedings of the 8th International Workshop on Satisfiability Modulo Theories (Edinburgh, UK)*, A. Gupta and D. Kroening, Eds., 2010.

[6] L. M. de Moura and N. Bjørner, "Z3: an efficient SMT solver," in *TACAS*, ser. Lecture Notes in Computer Science, vol. 4963. Springer, 2008, pp. 337–340.

[7] The Move Team, "The Move Programming Language," 2020. [Online]. Available: https://diem.github.io/move

[8] S. Blackshear, D. L. Dill, S. Qadeer, C. W. Barrett, J. C. Mitchell, O. Padon, and Y. Zohar, "Resources: A safe language abstraction for money," 2020. [Online]. Available: https://arxiv.org/abs/2004.05106

[9] H. Adams, N. Zinsmeister, and D. Robinson, "Uniswap v2 core," https://uniswap.org/whitepaper.pdf, Mar. 2020, accessed: 2026-04-21.

[10] K. R. M. Leino, "This is boogie 2," 2008, manuscript KRML 178. [Online]. Available: https://www.microsoft.com/en-us/research/publication/this-is-boogie-2-2/
[11] E. W. Dijkstra, "Guarded commands, nondeterminacy and formal derivation of programs," *Communications of the ACM*, vol. 18, no. 8, pp. 453–457, 1975.
[12] C. B. Jones, *Systematic Software Development Using VDM*, 2nd ed. Prentice-Hall, 1990.
[13] J. M. Spivey, *The Z Notation: A Reference Manual*, 2nd ed. Prentice-Hall, 1992.
[14] W. Grieskamp, "A computational model for Z based on concurrent constraint resolution," in *ZB 2000: Formal Specification and Development in Z and B*, ser. Lecture Notes in Computer Science, J. P. Bowen, S. Dunne, A. Galloway, and S. King, Eds., vol. 1878. Springer, 2000, pp. 414–432.
[15] J.-R. Abrial, *The B-Book: Assigning Programs to Meanings*. Cambridge University Press, 1996.
[16] A. Lattuada, T. Hance, C. Cho, M. Brun, I. Subasinghe, Y. Zhou, J. Howell, B. Parno, and C. Hawblitzel, "Verus: Verifying rust programs using linear ghost types," in *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, 2023, pp. 286–315.
[17] N. Swamy, C. Hriţcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoué, and S. Zanella-Béguelin, "Dependent types and multi-monadic effects in F*," in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 2016, pp. 256–270.
[18] K. Maillard, D. Ahman, R. Atkey, G. Martínez, C. Hritcu, E. Rivas, and E. Tanter, "Dijkstra monads for all," in *24th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2019. [Online]. Available: https://arxiv.org/abs/1903.01237
[19] K. M. Leino, "Accessible software verification with dafny," *IEEE Software*, vol. 34, no. 06, pp. 94–97, nov 2017.
[20] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Automated Deduction – CADE 28*, ser. Lecture Notes in Computer Science, A. Platzer and G. Sutcliffe, Eds., vol. 12699. Springer, 2021, pp. 625–635.
[21] T. Nipkow, L. C. Paulson, and M. Wenzel, *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, ser. Lecture Notes in Computer Science. Springer, 2002, vol. 2283.
[22] Y. Bertot and P. Castéran, *Interactive Theorem Proving and Program Development. Coq'Art: The Calculus of Inductive Constructions*, ser. Texts in Theoretical Computer Science. An EATCS Series. Springer, 2004.
[23] Certora, "The Certora Prover," https://www.certora.com/, 2024.
[24] L. Alt and C. Reitwießner, "SMT-Based Verification of Solidity Smart Contracts," in *ISoLA (4)*, ser. Lecture Notes in Computer Science, vol. 11247. Springer, 2018, pp. 376–388.
[25] Á. Hajdu and D. Jovanovic, "solc-verify: A modular verifier for solidity smart contracts," *CoRR*, vol. abs/1907.04262, 2019.
[26] ——, "SMT-Friendly Formalization of the Solidity Memory Model," in *ESOP*, ser. Lecture Notes in Computer Science, vol. 12075. Springer, 2020, pp. 224–250.

Appendix
AI-Based Development Approach

The implementation described in this paper was developed in close collaboration with AI coding assistants — primarily Claude Code, with Codex consulted as a second opinion on critical design decisions. The approach relied on a self-validating feedback loop: verification and specification inference (Sec. IV) were developed in parallel, so that any change to the encoding immediately had a baseline of test cases against which the AI could check itself — the inferred specs must verify end-to-end, with no human in the loop required to assess correctness.

What went well.

- *Specification language extensions in the frontend.* Parser and type-checker changes for behavioral predicates and state labels were largely produced by the AI, with human effort limited to review.
- *Bug identification and fixing.* Once a problematic example was unearthed — typically by inference producing a spec that failed verification — the AI was effective at diagnosing the root cause and proposing a principled fix.
- AI was very code at diagnosing verification problems like timeouts, by analyzing generated Boogie and SMT code, using Z3's logging features to observe quantifier instantiation, etc.
- *Boilerplate and well-understood areas.* Routine code such as expression simplification, AST traversals, and bytecode-level rewrites was handled essentially autonomously.

What went less well.

- *Hand-holding in the complex parts.* The Boogie encoding of behavioral predicates, the discrimination over closure variants, and the existential treatment of intermediate state labels all required substantial human guidance. The AI's first attempts were rarely correct for the general case, and judgement calls about soundness needed to be made by the human.
- *Many iterations.* Reaching a correct design often required dozens of iterations and significant elapsed time; in the most complex areas, the total effort approached that of manual implementation.
- *Token cost.* Long-running iterative work over a large codebase accumulates substantial inference cost.

Overall, the experience was positive. The validation pipeline gave the AI an unambiguous correctness signal independent of human review, and the AI carried a disproportionate share of the work in areas where the design space was already well understood. The harder, design-novel parts remained human-driven — but the AI made the surrounding work substantially less laborious.